

tf.space_to_batch Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/space_to_batch

SpaceToBatch for N-D tensors of type T.

Function Signatures

```
tf.space_to_batch( input, block_shape, paddings, name=None )
```

Code Examples

```
tf.space_to_batch(  
input, block_shape, paddings, name=None  
)
```

```
tf.space_to_batch(  
input, block_shape, paddings, name=None  
)
```

```
x = [[[[1], [2]], [[3], [4]]]]
```

```
x = [[[[1], [2]], [[3], [4]]]]
```

```
[[[[1]]], [[2]], [[3]], [[4]]]
```

```
[[[[1]]], [[2]], [[3]], [[4]]]
```

```
x = [[[[1, 2, 3], [4, 5, 6]],  
      [[7, 8, 9], [10, 11, 12]]]
```

```
x = [[[[1, 2, 3], [4, 5, 6]],  
      [[7, 8, 9], [10, 11, 12]]]
```

Documentation

SpaceToBatch for N-D tensors of type T.

View aliases

Main aliases

`tf.nn.space_to_batch`

This operation divides "spatial" dimensions $[1, \dots, M]$ of the input into a grid of blocks of shape `block_shape`, and interleaves these blocks with the "batch" dimension (0) such that in the output, the spatial dimensions $[1, \dots, M]$ correspond to the position within the grid, and the batch dimension combines both the position within a spatial block and the original batch position. Prior to division into blocks, the spatial dimensions of the input are optionally zero padded according to `padding`s. See below for a precise description.

This operation is equivalent to the following steps:

- Zero-pad the start and end of dimensions $[1, \dots, M]$ of the

input according to `padding`s to produce `padded` of shape `padded_shape`.

- Reshape `padded` to `reshaped_padded` of shape:

`[batch] +`
`[padded_shape[1] / block_shape[0],`
`block_shape[0],`
`...,`
`padded_shape[M] / block_shape[M-1],`
`block_shape[M-1]] +`
`remaining_shape`

- Permute dimensions of `reshaped_padded` to produce

`permuted_reshaped_padded` of shape:

`block_shape +`
`[batch] +`
`[padded_shape[1] / block_shape[0],`
`...,`
`padded_shape[M] / block_shape[M-1]] +`
`remaining_shape`

- Reshape `permuted_reshaped_padded` to flatten `block_shape` into the batch

dimension, producing an output tensor of shape:

`[batch * prod(block_shape)] +`
`[padded_shape[1] / block_shape[0],`
`...,`
`padded_shape[M] / block_shape[M-1]] +`
`remaining_shape`

Zero-pad the start and end of dimensions `[1, ..., M]` of the input according to paddings to produce `padded` of shape `padded_shape`.

Reshape `padded` to `reshaped_padded` of shape:

`[batch] +`
`[padded_shape[1] / block_shape[0],`
`block_shape[0],`
`...,`

padded_shape[M] / block_shape[M-1],
block_shape[M-1]] +
remaining_shape

Permute dimensions of reshaped_padded to produce
permuted_reshaped_padded of shape:

block_shape +
[batch] +
[padded_shape[1] / block_shape[0],
...,
padded_shape[M] / block_shape[M-1]] +
remaining_shape

Reshape permuted_reshaped_padded to flatten block_shape into the batch
dimension, producing an output tensor of shape:

[batch * prod(block_shape)] +
[padded_shape[1] / block_shape[0],
...,
padded_shape[M] / block_shape[M-1]] +
remaining_shape

Some examples:

(1) For the following input of shape [1, 2, 2, 1], block_shape = [2, 2], and
paddings = [[0, 0], [0, 0]]:

The output tensor has shape [4, 1, 1, 1] and value:

(2) For the following input of shape [1, 2, 2, 3], block_shape = [2, 2], and

`paddings = [[0, 0], [0, 0]]:`

The output tensor has shape `[4, 1, 1, 3]` and value:

(3) For the following input of shape `[1, 4, 4, 1]`, `block_shape = [2, 2]`, and `paddings = [[0, 0], [0, 0]]:`

The output tensor has shape `[4, 2, 2, 1]` and value:

(4) For the following input of shape `[2, 2, 4, 1]`, `block_shape = [2, 2]`, and `paddings = [[0, 0], [2, 0]]:`

The output tensor has shape `[8, 1, 3, 1]` and value:

Among others, this operation is useful for reducing atrous convolution into regular convolution.

Returns